

MATH70097 - Supervised Learning: Assessment 1

A technology company is running an online advertising campaign. Users are shown a banner while browsing the web. Some users click on the banner and then subscribe to the service. This event is called a conversion. Your task is to predict whether a conversion occurs for each user, based on information about: • the user, • the current advertising campaign, • and a previous related advertising campaign. You will build a predictive model using historical data and use it to make predictions for new users.

Pulling from the lecture notes and assessment brief, we approach this assignment in 4 parts.

1. **Exploratory Data Analysis and Preprocessing** — understand the structure and signals, detect issues (missing values, unusual values, imbalance, nonlinearity).
2. **Building Prediction Models** — train and compare Logistic Regression (with Ridge and Lasso variants), kNN, LDA, QDA, and Naive Bayes.
3. **Validation and Model Selection** — 10-fold stratified CV for hyperparameter tuning, Bootstrap for uncertainty quantification, and principled model selection via log-loss.
4. **Generating Test Predictions** — refit on the full training set and produce predicted probabilities.

Setup & Loading data

We import the relevant packages, set a seed, and load the data. The kernel used is the ISLP (Python 3.12.2) Kernel - which we setup in week 1 of the course.

```
In [67]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import StratifiedKFold, cross_val_score, cross_val_predict
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import log_loss, roc_auc_score, roc_curve, confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.calibration import calibration_curve
from sklearn.decomposition import PCA
from sklearn.base import clone
import warnings
warnings.filterwarnings('ignore')
```

```
sns.set_theme(style='whitegrid', palette='muted')
pd.set_option('display.max_columns', 25)
SEED = 42
np.random.seed(SEED)
```

```
In [68]: train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')
print(f"Training set : {train.shape[0]} rows, {train.shape[1]} columns")
print(f"Test set      : {test.shape[0]} rows, {test.shape[1]} columns")
train.head()
```

```
Training set : 31647 rows, 17 columns
Test set      : 13564 rows, 17 columns
```

```
Out[68]:
```

	age	job	marital	education	X2	X4	X3	X1	device	day	month
0	36	technician	single	tertiary	0	0	0	0	unknown	17	jur
1	56	entrepreneur	married	secondary	0	196	0	0	cellular	19	nov
2	46	blue-collar	married	secondary	0	0	1	0	unknown	5	jur
3	41	management	divorced	tertiary	0	3426	0	0	cellular	1	oct
4	38	blue-collar	married	secondary	0	0	1	0	unknown	20	may

Part 1: Explore the training data (EDA)

Following the data descriptions from the brief, as well as looking through our dataframe, we choose to split the variables into two types. Quantitative variables and Qualitative (categorical) variables.

```
In [69]: # Define quantitative variables
quantitative = ['age', 'X4', 'day', 'time_spent', 'banner_views', 'days_e

# Define qualitative variables
qualitative = ['job', 'marital', 'education', 'X1', 'X2', 'X3', 'device',

# Print qualitative and quantitative variables

print("Quantitative variables:")
print(train[quantitative].dtypes)
print("\nQualitative variables:")
for col in qualitative:
    levels = train[col].unique()
    print(f"{col} : {len(levels)} levels - {list(levels)}")

train.describe()
```

Quantitative variables:

```
age          int64
X4           int64
day          int64
time_spent   int64
banner_views int64
days_elapsed_old float64
banner_views_old int64
dtype: object
```

Qualitative variables:

```
job : 12 levels - ['technician', 'entrepreneur', 'blue-collar', 'managemen
t', 'admin.', 'retired', 'housemaid', 'unknown', 'services', 'student', 'u
nemployed', 'self-employed']
marital : 3 levels - ['single', 'married', 'divorced']
education : 4 levels - ['tertiary', 'secondary', 'primary', 'unknown']
X1 : 2 levels - [np.int64(0), np.int64(1)]
X2 : 2 levels - [np.int64(0), np.int64(1)]
X3 : 2 levels - [np.int64(0), np.int64(1)]
device : 3 levels - ['unknown', 'cellular', 'telephone']
outcome_old : 4 levels - ['unknown', 'success', 'other', 'failure']
month : 12 levels - ['jun', 'nov', 'oct', 'may', 'jul', 'aug', 'feb', 'dec
', 'sep', 'jan', 'mar', 'apr']
```

Out[69]:

	age	X2	X4	X3	X1	
count	31647.000000	31647.000000	31647.000000	31647.000000	31647.000000	3164
mean	40.941669	0.019370	1357.985465	0.556040	0.160458	1
std	10.632010	0.137824	2976.874443	0.496857	0.367036	
min	18.000000	0.000000	-6847.000000	0.000000	0.000000	
25%	33.000000	0.000000	70.000000	0.000000	0.000000	
50%	39.000000	0.000000	446.000000	1.000000	0.000000	1
75%	48.000000	0.000000	1434.000000	1.000000	0.000000	2
max	95.000000	1.000000	81204.000000	1.000000	1.000000	3

We find that there are no null or NaN values in the dataset. However, for 'job', 'education', 'device' and 'outcome_old', we have "unknown" values to serve as a missing indicator.

Similarly, for 'days_elapsed_old', we have values -1, indicating that the user did not see the old banner.

```
In [70]: print("NaN/Null values in training set:")
print(train.isnull().sum())
```

NaN/Null values in training set:

```
age          0
job          0
marital      0
education    0
X2           0
X4           0
X3           0
X1           0
device       0
day          0
month        0
time_spent   0
banner_views 0
days_elapsed_old 0
banner_views_old 0
outcome_old  0
y            0
dtype: int64
```

```
In [71]: print("unknown values in training set:")
        for x in ['job', 'education', 'device', 'outcome_old']:
            unknown_count = (train[x] == 'unknown').sum()
            percentage = (unknown_count / len(train)) * 100
            print(f"{x} : {unknown_count} ({percentage:.2f}%)")
```

unknown values in training set:

```
job : 203 (0.64%)
education : 1322 (4.18%)
device : 9065 (28.64%)
outcome_old : 25867 (81.74%)
```

```
In [72]: print("No. of days_elapsed_old = -1 in training set:")
        print((train['days_elapsed_old'] == -1).sum())

        print("No. of days_elapsed_old = -1 in test set:")
        print((test['days_elapsed_old'] == -1).sum())
```

No. of days_elapsed_old = -1 in training set:

0

No. of days_elapsed_old = -1 in test set:

0

Missing values takeaway

"unknown": we can retain "unknown" as a factor level. The lack of a value itself provides us with information on users which were not exposed to the old banner

'device': over 80% of the values are "unknown". We keep the variable but approach it carefully in model development as it carries limited signal to our predictors.

'days_elapsed_old': No -1 values are found in our dataset, so we can avoid handling this case specifically as we don't have any test or train data for it.

Numeric Variables: There are no missing numerical values, so we do not have to perform specific imputation (filling).

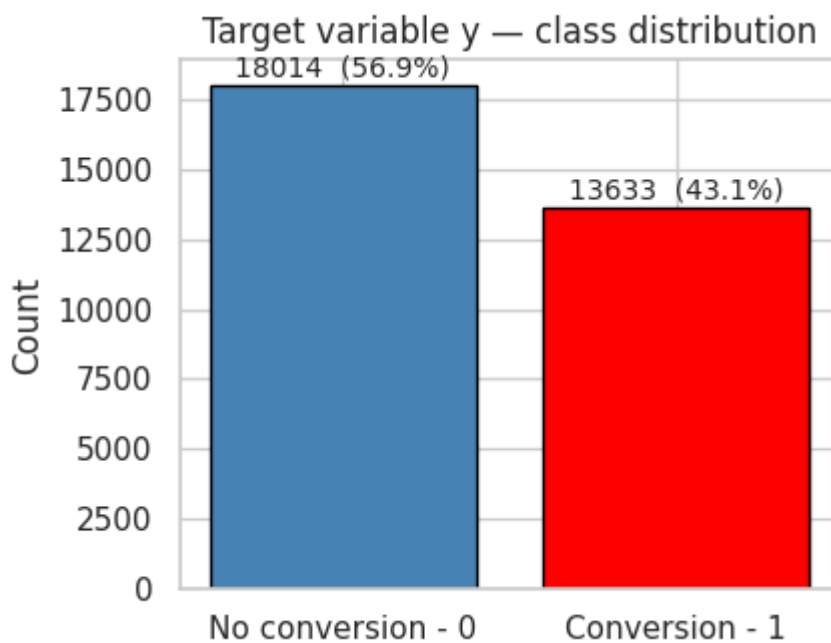
After checking for missing and unusual values, we check for imbalances.

```
In [73]: rate = train['y'].mean()
print(f"Overall conversion rate (y=1) in training set: {rate:.4f}")
ratio = (train['y'] == 0).sum() / (train['y'] == 1).sum()
print(f"Class imbalance ratio (y=0 : y=1) in training set: {ratio:.2f} :

fig, ax = plt.subplots(figsize=(4.5, 3.5))
counts = train['y'].value_counts().sort_index()
bars = ax.bar(['No conversion - 0', 'Conversion - 1'], counts.values,
              color=['steelblue', 'red'], edgecolor='black')
for i, v in enumerate(counts.values):
    ax.text(i, v + 300, f"{v} ({100*v/len(train):.1f}%)", ha='center', f
ax.set_ylabel('Count')
ax.set_title('Target variable y - class distribution')
plt.tight_layout()
plt.show()
```

Overall conversion rate (y=1) in training set: 0.4308

Class imbalance ratio (y=0 : y=1) in training set: 1.32 : 1



Imbalances takeaway

The targeted split has a mild imbalance, 57% non-conversion to 43% conversion. We shouldn't use specific resampling techniques here, but we do focus on a couple of key points:

1. Use stratified cross-validation to preserve class proportions in every fold.
2. Keep the baseline in mind: a naïve classifier predicting the majority class achieves only ~57% accuracy.

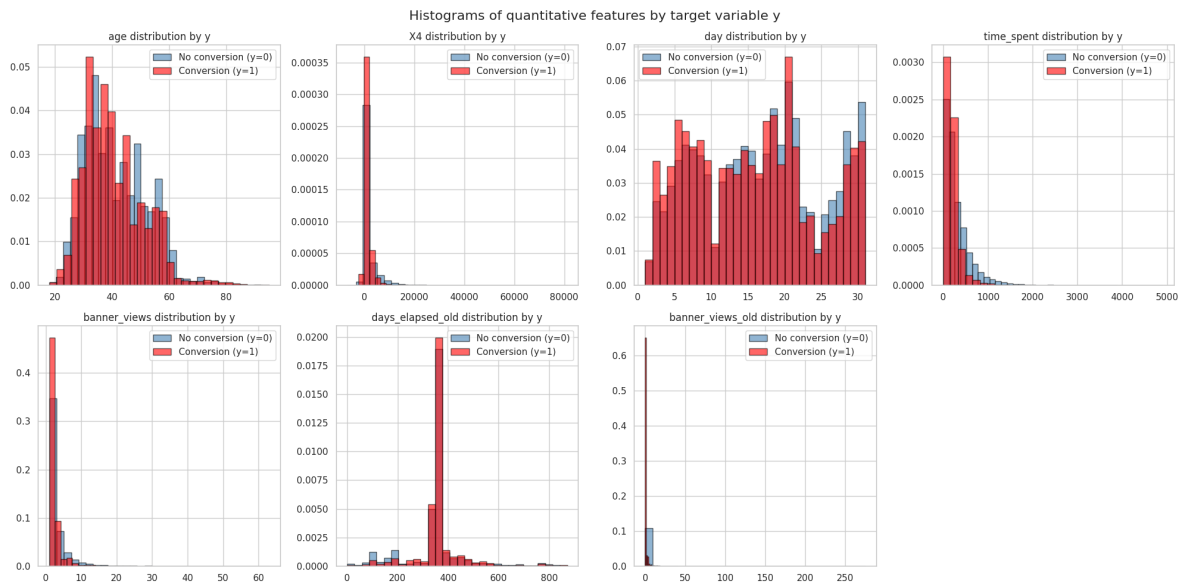
We proceed to visualise structure:

```
In [74]: fig, axes = plt.subplots(2, 4, figsize=(20, 10))
axes = axes.ravel()

for i, col in enumerate(quantitative):
    ax = axes[i]
    for yval, colour, label in [(0, 'steelblue', 'No conversion (y=0)'),
                                (1, 'red', 'Conversion (y=1)')]:
```

```
subset = train.loc[train['y'] == yval, col]
ax.hist(subset, bins=30, alpha=0.6, color=colour, label=label, ed
ax.set_title(f"{col} distribution by y")
ax.legend()
```

```
axes[-1].set_visible(False)
plt.suptitle("Histograms of quantitative features by target variable y",
plt.tight_layout()
plt.show()
```



Quantitative structure takeaways

We find a couple of key observations that may help us out:

X4 is heavily right-skewed, so a log-transform may help with our linear-models.

banner_views and banner_views_old are right-skewed.

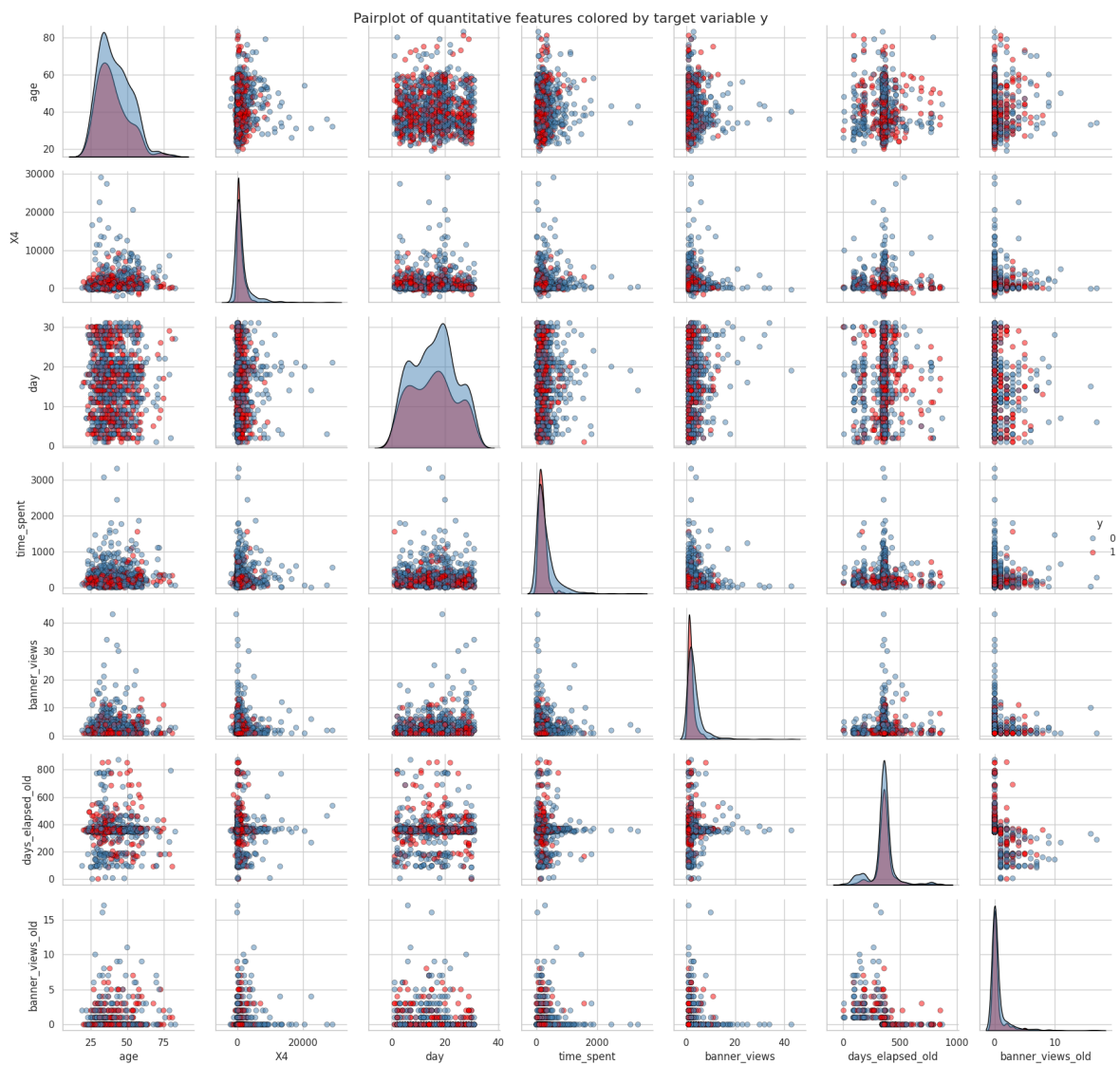
time_spent interestingly shows a significant shift between classes. Non-conversion (y=0) spend more time. This seems to signal that quick engagement and attention-capture has a larger chance of conversion.

age is (roughly) symmetric, and has some class separation.

We move onto exploring quantitative variables

```
In [75]: vars = quantitative
sample = train[vars + ['y']].sample(1000, random_state=SEED)
sample['y'] = sample['y'].astype(str)

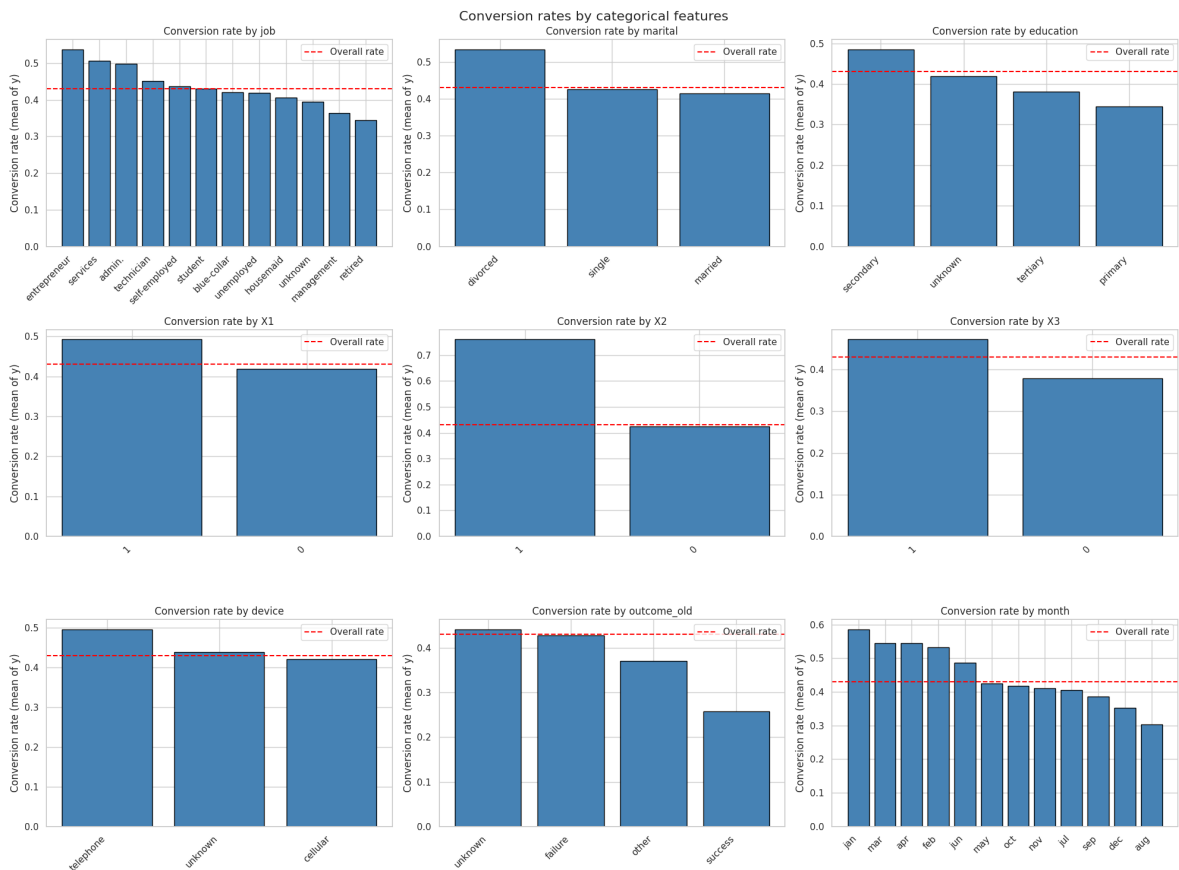
g = sns.pairplot(sample, hue='y', vars=vars, diag_kind='kde',
                  palette={'0': 'steelblue', '1': 'red'},
                  plot_kws={'alpha': 0.5, 'edgecolor': 'black'},
                  diag_kws={'alpha': 0.5, 'edgecolor': 'black'})
g.fig.suptitle("Pairplot of quantitative features colored by target varia
plt.tight_layout()
plt.show()
```



Our pairplot reveals substantial overlaps. No single feature separates conversion rate cleanly.

We move onto conversion rate by categorical variables

```
In [76]: cat_vars = qualitative
fig, axes = plt.subplots(3, 3, figsize=(20, 15))
for ax, col in zip(axes.ravel(), cat_vars):
    rates = train.groupby(col)['y'].agg(['mean', 'count']).sort_values('mean')
    bars = ax.bar(range(len(rates)), rates['mean'], color='steelblue', edgecolor='black')
    ax.set_xticks(range(len(rates)))
    ax.set_xticklabels(rates.index, rotation=45, ha='right')
    ax.set_title(f"Conversion rate by {col}")
    ax.set_ylabel("Conversion rate (mean of y)")
    ax.axhline(train['y'].mean(), color='red', linestyle='--', label='Overall Mean')
    ax.legend()
plt.suptitle("Conversion rates by categorical features", fontsize=16)
plt.tight_layout()
plt.show()
```



Key Qualitative (categorical) takeaways:

outcome_old: "success" has the lowest conversion rate, around 26%.

month: suggests seasonal variation, with Jan, Mar, April being the highest.

Job: Students and Retirees convert at the largest rates.

We proceed to check for correlation structure and nonlinearity

```
In [77]: # Pearson correlation of quantitative features with target variable y
corr = train[quantitative + ['y']].corr().drop('y').sort_values('y', ascending=False)
print("Pearson correlation of target variable y:")
for var, val in corr['y'].items():
    bar = '█' * int(abs(val) * 50)
    sign = '+' if val > 0 else '-'
    print(f"{var:20s} : {val:.4f} {sign} {bar}")
```

Pearson correlation of target variable y:

```
time_spent      : -0.2009 - ██████████
banner_views    : -0.1687 - ██████████
X4              : -0.0778 - ████████
day            : -0.0639 - ████████
age            : -0.0531 - ████████
banner_views_old : -0.0224 - ████████
days_elapsed_old : 0.0542 + ████████
```

```
In [78]: # Nonlinearity check

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

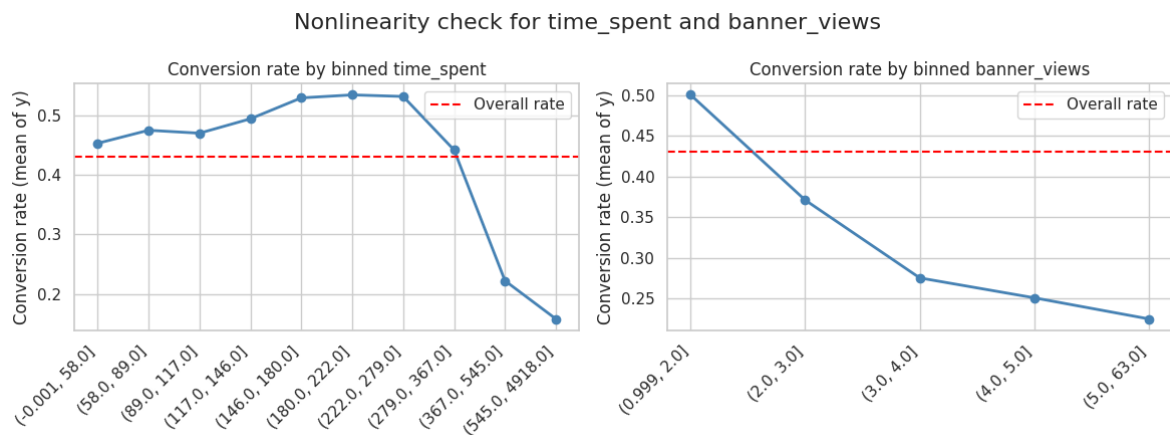
for ax, col in zip(axes, ['time_spent', 'banner_views']):
    helper = train.copy()
```



```

helper[col + '_bin'] = pd.qcut(helper[col], q=10, duplicates='drop')
rates = helper.groupby(col + '_bin')['y'].mean()
ax.plot(range(len(rates)), rates.values, 'o-', color='steelblue', lin
ax.set_xticks(range(len(rates)))
ax.set_xticklabels(rates.index.astype(str), rotation=45, ha='right')
ax.set_title(f"Conversion rate by binned {col}")
ax.set_ylabel("Conversion rate (mean of y)")
ax.axhline(train['y'].mean(), color='red', linestyle='--', label='Over
ax.legend()
plt.suptitle("Nonlinearity check for time_spent and banner_views", fontsi
plt.tight_layout()
plt.show()

```



time_spent and banner_views show decreasing relationships that are monotonic with conversion. This pattern indicates that logistic regression and LDA may be suitable models. On the tails(extremities) we have curvature, which indicates that kNN and QDA may be a reasonable signal capture in the tails.

Dimensionality discussion

The curse of dimensionality is an important theoretical consideration. The number of predictors p is large relative to the structure of the data, expanding our dimensionality.

```

In [79]: # Dimensionality count after one-hot encoding
var_counts = ['job', 'marital', 'education', 'device', 'outcome_old', 'mo
levels = sum(train[var].nunique() for var in var_counts)

dummies = sum(train[var].nunique() - 1 for var in var_counts)

n_numeric = len(quantitative)
n_binary = 3
n_engineered = 4

p_total = n_numeric + dummies + n_binary + n_engineered
n = len(train)

print(f"Total features after one-hot encoding: {p_total}")
print(f"Number of training samples: {n}")
print(f"n/p ratio: {n/p_total:.0f}")

```

Total features after one-hot encoding: 46
Number of training samples: 31647
n/p ratio: 688

The n/p ratio is comfortable for parametric models at 688, but the expanded dimensionality has consequences for kNN:

The nearest neighbour degrades. As data points become approximately equidistant, the query point no longer remains local. This leads to kNN bias increasing in average dimensions, as it averages over points that are not close, and so can no longer effectively capture local structure. This leads to a large k to stabilise predictions. We may find that Lasso logistic regression or Naive bayes work better here.

Feature encoding takeaways

We define the processing pipeline across all models tested, as such:

Quantitative variables - standardised using StandardScaler for kNN and regularised logistical regression.

Qualitative variables - Converted to dummy variables using one-hot encoding. First level is dropped to avoid multicollinearity, and "unknown" is retained as a useful level.

We proceed with feature engineering:

```
In [80]: def engineer_features(df):
    df = df.copy()
    df['old_campaign_exposed'] = (df['banner_views_old'] > 0).astype(int)
    df['old_success'] = (df['outcome_old'] == 'success').astype(int)
    df['time_per_view'] = df['time_spent'] / (df['banner_views'] + 1).clip(
    df['X4_log'] = np.sign(df['X4']) * np.log1p(np.abs(df['X4'])) # Log-
    return df

train_featured = engineer_features(train)
test_featured = engineer_features(test)

print("New features added: old_campaign_exposed, old_success, time_per_vi
New features added: old_campaign_exposed, old_success, time_per_view, X4_l
og
```

```
In [81]: numeric_features = quantitative + ['time_per_view', 'X4_log']
binary_features = ['old_campaign_exposed', 'old_success', 'X1', 'X2', 'X3']
categorical_features = ['job', 'marital', 'education', 'device', 'outcome']

all_features = numeric_features + binary_features + categorical_features
print(f"Total features after engineering: {len(all_features)}")

x_train = train_featured[all_features]
y_train = train_featured['y']
x_test = test_featured[all_features]
test_ids = test_featured['ID']

# Preprocessor pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('bin', 'passthrough', binary_features),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), categorical_features)
```

```

    ]
)

print("Preprocessor pipeline created with standard scaling for numeric fe
print(f"feature breakdown: {len(numeric_features)} numeric, {len(binary_f
print(f"X_train shape before preprocessing: {x_train.shape}, X_test shape

```

Total features after engineering: 20

Preprocessor pipeline created with standard scaling for numeric features, passthrough for binary features, and one-hot encoding for categorical features.

feature breakdown: 9 numeric, 5 binary, 6 categorical (one-hot encoded to 32 features)

X_train shape before preprocessing: (31647, 20), X_test shape before preprocessing: (13564, 20)

PCA Analysis

PCA Analysis was introduced in week 4 as an unsupervised tool to find maximal variance directions. We apply it to check if the data admits a low-dimensional structure, and visualises class separation in the reduced spaces.

```

In [82]: X_train_processed = preprocessor.fit_transform(x_train)
print(f"X_train shape after preprocessing: {X_train_processed.shape}")

pca = PCA(n_components=10, random_state=SEED)
X_pca = pca.fit_transform(X_train_processed)
print(f"Shape of PCA-transformed training data: {X_pca.shape}")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

ax = axes[0]
cum_var = np.cumsum(pca.explained_variance_ratio_) * 100
ax.bar = ax.bar(range(1, len(cum_var) + 1), cum_var, color='steelblue', e
ax.set_xticks(range(1, len(cum_var) + 1))
ax.set_xticklabels(range(1, len(cum_var) + 1))
ax.set_xlabel('Number of principal components')
ax.set_ylabel('Cumulative explained variance (%)')
ax.set_title('PCA scree plot - Explained Variance')

ax = axes[1]
colours = y_train.map({0: 'steelblue', 1: 'red'})
sample_idx = np.random.choice(len(X_pca), size=3000, replace=False)
ax.scatter(X_pca[sample_idx, 0], X_pca[sample_idx, 1], c=colours.iloc[sam
ax.set_xlabel('Principal Component 1')
ax.set_ylabel('Principal Component 2')
ax.set_title('PCA scatter plot - first 2 components')

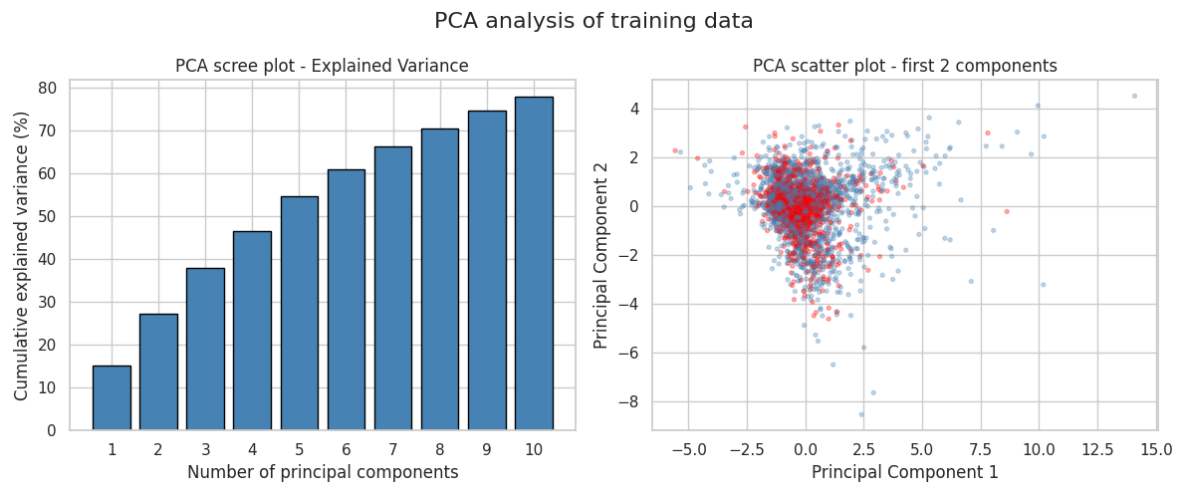
plt.suptitle("PCA analysis of training data", fontsize=16)
plt.tight_layout()
plt.show()

print(f"Cumulative variance explained by 5 PCs: {cum_var[4]:.1f}%")
print(f"Cumulative variance explained by 10 PCs: {cum_var[9]:.1f}%")

```

X_train shape after preprocessing: (31647, 52)

Shape of PCA-transformed training data: (31647, 10)



Cumulative variance explained by 5 PCs: 54.5%

Cumulative variance explained by 10 PCs: 77.8%

The PCA Scree plot confirms the high-effective dimensionality, there is no clear separator in low dimensional projections.

EDA Recap

Moderate class imbalance (43% positive): Use stratified CV; log-loss as evaluation metric.

"unknown" encodes structural missingness: Retain as its own category, do not impute.

No literal NaN in numeric columns: No numeric imputation needed.

outcome_old = "success" has the lowest conversion: Strong predictor; include as feature.

time_spent, banner_views negatively correlated with y: Key numeric predictors with a near-linear relationship.

X4 heavily skewed: Apply log-transform.

month, X1 show large group-level differences: Important categorical predictors.

device` mostly unknown: Low signal so we can include but expect low importance.

High effective dimensionality (~40 after one-hot): Curse of dimensionality affects kNN; favours Lasso for variable selection and Naive Bayes for variance reduction.

Pair plot / PCA show overlap, no dramatic nonlinearity: Linear models (LR, LDA) should be competitive

Step 2 - Model Building and Justification

We choose to train 6 models, covered by the course. We see no reason to combine models, as EDA does not strongly suggest a benefit. All models are wrapped in a pipeline with consistent preprocessing. Models are listed as follows:

1. Logistic regression (Ridge/L2) - directly estimates $P(y=1 | X)$ with L2 regularisation.
2. Logistic regression (Lasso/L1) - same as above but with an L1 penalty, performing automatic variable selection by shrinking irrelevant coefficients to 0. Useful due to unnamed and dummy variables.
3. k-nearest neighbours - non-parametric, requires standardised features.
4. Linear Discriminant Analysis - assumes each class has a multivariate gaussian distribution with a shared covariance matrix.
5. Quadratic Discriminant Analysis - similar to LDA, but allows each class to have its own covariance matrix, leading to flexibility at the cost of more estimate parameters.
6. Naive Bayes - assumes predictors are independent given the class. This "naive" assumption effectively reduces a p -dimensional estimation problem to p univariate problems, dramatically reducing variance. This makes it well-suited to the high dimensionality after one-hot encoding, at the cost of potentially increased bias if predictors are correlated.

```
In [83]: cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=SEED)

models = {
    'Logistic Regression (Ridge, C=1)': Pipeline([
        ('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l2', C=1.0, max_iter=2000,
                                   solver='lbfgs', random_state=SEED))
    ]),
    'Logistic Regression (Lasso, C=1)': Pipeline([
        ('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l1', C=1.0, max_iter=2000,
                                   solver='saga', random_state=SEED))
    ]),
    'kNN (k=30)': Pipeline([
        ('pre', preprocessor),
        ('clf', KNeighborsClassifier(n_neighbors=30))
    ]),
    'LDA': Pipeline([
        ('pre', preprocessor),
        ('clf', LinearDiscriminantAnalysis())
    ]),
    'QDA': Pipeline([
        ('pre', preprocessor),
        ('clf', QuadraticDiscriminantAnalysis(reg_param=0.5))
    ]),
    'Naive Bayes': Pipeline([
        ('pre', preprocessor),
        ('clf', GaussianNB())
    ]),
}

print(f"{'Model':<40s} {'Log-Loss':>18s} {'ROC-AUC':>18s}")
print('-' * 80)

results = {}
for name, pipe in models.items():
    ll_scores = cross_val_score(pipe, x_train, y_train, cv=cv,
                                scoring='neg_log_loss', n_jobs=-1)
```

```

auc_scores = cross_val_score(pipe, x_train, y_train, cv=cv,
                              scoring='roc_auc', n_jobs=-1)

results[name] = {
    'log_loss_mean': -ll_scores.mean(),
    'log_loss_std':   ll_scores.std(),
    'auc_mean':       auc_scores.mean(),
    'auc_std':        auc_scores.std(),
}
print(f"{name:<40s} {-ll_scores.mean():.4f} ± {ll_scores.std():.4f}
      f"{auc_scores.mean():.4f} ± {auc_scores.std():.4f}")

```

Model	Log-Loss		R
OC-AUC			

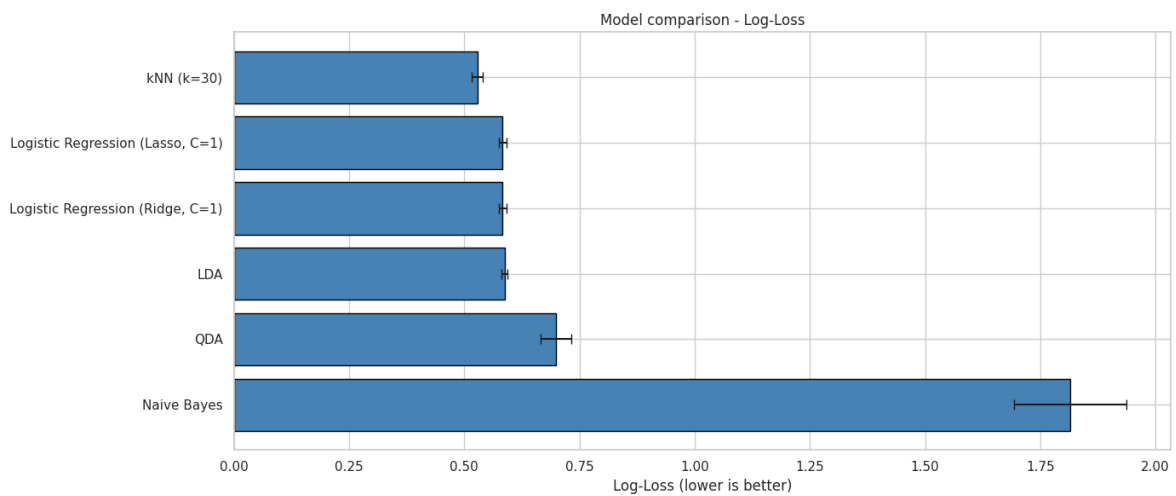
Logistic Regression (Ridge, C=1)	0.5834 ± 0.0085	0.7597 ± 0.010	
0			
Logistic Regression (Lasso, C=1)	0.5833 ± 0.0085	0.7598 ± 0.010	
0			
kNN (k=30)	0.5281 ± 0.0123	0.8174 ± 0.010	
1			
LDA	0.5872 ± 0.0071	0.7524 ± 0.010	
2			
QDA	0.6983 ± 0.0334	0.7682 ± 0.010	
8			
Naive Bayes	1.8148 ± 0.1212	0.7059 ± 0.011	
5			

```

In [86]: res_df = pd.DataFrame(results).T.sort_values('log_loss_mean')

fig, ax = plt.subplots(figsize=(14, 6))
ax.barh(res_df.index, res_df['log_loss_mean'], xerr=res_df['log_loss_std',
    color='steelblue', edgecolor='black', capsize=4)
ax.set_xlabel('Log-Loss (lower is better)')
ax.set_title('Model comparison - Log-Loss')
ax.invert_yaxis()
plt.tight_layout()
plt.show()

```



Initial comparison takeaways:

1. Ridge and Lasso Logistic regression perform similarly at C = 1. We will tune for C for each separately.
2. LDA compares to logistic regression, as expected as we estimate similar linear

boundaries.

3. Naive Bayes has a significant log-loss, indicating that bias may be introduced as predictors are correlated (time_spent, banner views)
4. QDA seems to struggle with high-dimensional one-hot encoded features
5. KNN has not been tuned yet - its performance depends on k.

Step 3 - Validation and model selection

We use a 10-fold stratified cross-validation to tune hyperparameters, with the primary metric being log-loss to select models. We also use Bootstrao to quantify uncertainty for our performance metrics and assess whether differences are statistically meaningful.

Logistic Regression - Ridge tuning:

```
In [87]: C_values = [0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]
ridge_results = []

for C in C_values:
    pipe = Pipeline([
        ('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l2', C=C, max_iter=2000,
                                   solver='lbfgs', random_state=SEED))
    ])
    scores = cross_val_score(pipe, x_train, y_train, cv=cv,
                              scoring='neg_log_loss', n_jobs=-1)
    ridge_results.append({
        'C': C,
        'log_loss_mean': -scores.mean(),
        'log_loss_std': scores.std()
    })
    print(f"Ridge Logistic Regression (C={C}) - log-loss: {-scores.mean()}

ridge_df = pd.DataFrame(ridge_results)
best_C_ridge = ridge_df.loc[ridge_df['log_loss_mean'].idxmin(), 'C']
best_ll_ridge = ridge_df['log_loss_mean'].min()
print(f"\nBest Ridge Logistic Regression - C={best_C_ridge}, log-loss = {

Ridge Logistic Regression (C=0.001) - log-loss: 0.6036 ± 0.0061
Ridge Logistic Regression (C=0.01) - log-loss: 0.5863 ± 0.0081
Ridge Logistic Regression (C=0.05) - log-loss: 0.5836 ± 0.0084
Ridge Logistic Regression (C=0.1) - log-loss: 0.5834 ± 0.0085
Ridge Logistic Regression (C=0.5) - log-loss: 0.5834 ± 0.0085
Ridge Logistic Regression (C=1.0) - log-loss: 0.5834 ± 0.0085
Ridge Logistic Regression (C=5.0) - log-loss: 0.5834 ± 0.0085
Ridge Logistic Regression (C=10.0) - log-loss: 0.5834 ± 0.0085
Ridge Logistic Regression (C=50.0) - log-loss: 0.5834 ± 0.0085
Ridge Logistic Regression (C=100.0) - log-loss: 0.5834 ± 0.0085
```

Best Ridge Logistic Regression - C=0.5, log-loss = 0.5834

Logistic Regression - Lasso Tuning:

```
In [88]: lasso_results = []

for C in C_values:
```

```

pipe = Pipeline([
    ('pre', preprocessor),
    ('clf', LogisticRegression(penalty='l1', C=C, max_iter=2000,
                               solver='saga', random_state=SEED))
])
scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_
lasso_results.append({
    'C': C,
    'log_loss_mean': -scores.mean(),
    'log_loss_std': scores.std()
})
print(f"Lasso Logistic Regression (C={C}) - log-loss: {-scores.mean()}

lasso_df = pd.DataFrame(lasso_results)
best_C_lasso = lasso_df.loc[lasso_df['log_loss_mean'].idxmin(), 'C']
best_ll_lasso = lasso_df['log_loss_mean'].min()
print(f"\nBest Lasso Logistic Regression - C={best_C_lasso}, log-loss = {

```

```

Lasso Logistic Regression (C=0.001) - log-loss: 0.6456 ± 0.0033
Lasso Logistic Regression (C=0.01) - log-loss: 0.5939 ± 0.0077
Lasso Logistic Regression (C=0.05) - log-loss: 0.5839 ± 0.0085
Lasso Logistic Regression (C=0.1) - log-loss: 0.5834 ± 0.0085
Lasso Logistic Regression (C=0.5) - log-loss: 0.5833 ± 0.0085
Lasso Logistic Regression (C=1.0) - log-loss: 0.5833 ± 0.0085
Lasso Logistic Regression (C=5.0) - log-loss: 0.5834 ± 0.0085
Lasso Logistic Regression (C=10.0) - log-loss: 0.5834 ± 0.0085
Lasso Logistic Regression (C=50.0) - log-loss: 0.5834 ± 0.0085
Lasso Logistic Regression (C=100.0) - log-loss: 0.5834 ± 0.0085

```

```

Best Lasso Logistic Regression - C=0.5, log-loss = 0.5833

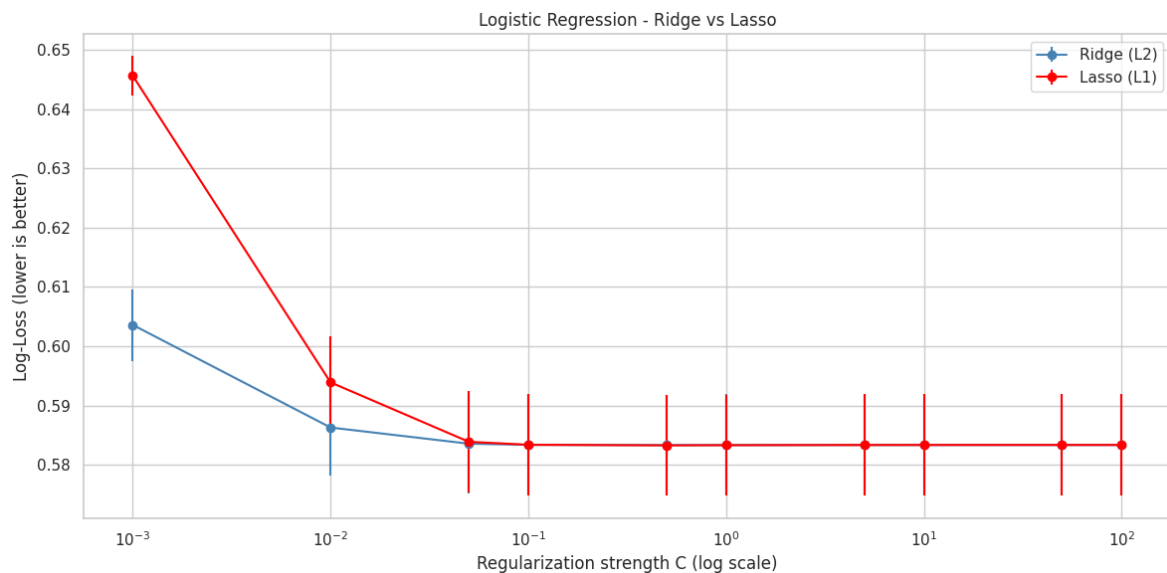
```

In [90]: *# Compare Ridge and Lasso results*

```

fig, ax = plt.subplots(figsize=(12, 6))
ax.errorbar(ridge_df['C'], ridge_df['log_loss_mean'], yerr=ridge_df['log_
ax.errorbar(lasso_df['C'], lasso_df['log_loss_mean'], yerr=lasso_df['log_
ax.set_xscale('log')
ax.set_xlabel('Regularization strength C (log scale)')
ax.set_ylabel('Log-Loss (lower is better)')
ax.set_title('Logistic Regression - Ridge vs Lasso')
ax.legend()
plt.tight_layout()
plt.show()

```

Model does not seem overtly sensitive to regularisation. If the best log-loss results are similar, Lasso is preferred for its variable selection properties, leading to a more interpretable model.

We continue to check which coefficients shrunk to 0 for the best C for the Lasso model.

```
In [93]: # Fit lasso on best C on full training data
lasso_inspect = Pipeline([
    ('pre', preprocessor),
    ('clf', LogisticRegression(penalty='l1', C=best_C_lasso, max_iter=200,
                              solver='saga', random_state=SEED))
])
lasso_inspect.fit(x_train, y_train)

# Get feature names after preprocessing
ohe = lasso_inspect.named_steps['pre'].named_transformers_['cat']
cat_feature_names = ohe.get_feature_names_out(categorical_features).tolist()
feature_names = numeric_features + binary_features + cat_feature_names

lasso_coefs = pd.Series(lasso_inspect.named_steps['clf'].coef_[0], index=
n_zero = (lasso_coefs == 0).sum()
n_nonzero = (lasso_coefs != 0).sum()

print(f"Total features after preprocessing: {len(feature_names)}")
print(f"Number of features with zero coefficients: {n_zero}")
print(f"Number of features with non-zero coefficients: {n_nonzero}")
print(("Features removed by Lasso (zero coefficients):\n"
      + "\n".join(lasso_coefs[lasso_coefs == 0].index.tolist())))

print("Survival of features with non-zero coefficients in Lasso:")
surviving_features = lasso_coefs[lasso_coefs != 0].reindex(
    lasso_coefs[lasso_coefs != 0].abs().sort_values(ascending=False).index)
for feat, coef in surviving_features.items():
    print(f"{feat:40s} : {coef:.4f}")
```

```
Total features after preprocessing: 52
Number of features with zero coefficients: 3
Number of features with non-zero coefficients: 49
Features removed by Lasso (zero coefficients):
job_technician
job_unknown
month_oct
Survival of features with non-zero coefficients in Lasso:
X2                : 2.2372
time_spent        : -0.9016
banner_views      : -0.5902
month_mar         : 0.5135
month_jan         : 0.5109
month_aug         : -0.5068
X3                : 0.4761
month_nov         : -0.4630
X4_log            : 0.4339
device_telephone  : 0.4264
job_entrepreneur  : 0.4245
device_unknown    : -0.4130
X4                : -0.4089
education_primary : -0.3984
old_campaign_exposed : -0.3894
marital_divorced  : 0.3762
month_may         : -0.3602
month_jul         : -0.3426
marital_single    : -0.2997
X1                : 0.2773
month_apr         : 0.2668
job_management    : -0.2541
month_jun         : 0.2395
job_retired       : -0.2221
old_success       : -0.2107
outcome_old_success : -0.2107
time_per_view     : 0.2081
education_secondary : 0.2079
marital_married   : -0.1905
outcome_old_other  : -0.1537
month_feb         : 0.1453
job_student       : -0.1413
outcome_old_unknown : 0.1353
job_unemployed    : -0.1329
device_cellular   : -0.1273
job_services      : 0.1235
age               : -0.1093
job_blue-collar   : -0.0859
day               : -0.0763
education_tertiary : -0.0714
days_elapsed_old  : 0.0694
education_unknown  : 0.0417
month_dec         : -0.0399
outcome_old_failure : 0.0231
banner_views_old   : 0.0213
month_sep         : -0.0189
job_self-employed : 0.0148
job_admin.        : 0.0136
job_housemaid     : 0.0093
```

Tuning kNN

The choice of k directly controls the bias-variance trade-off:

- Small k leads to low bias, high variance (overfitting risk).
- Large k leads to high bias, low variance (underfitting risk).

Given the curse of dimensionality discussion in Section 1.5, we expect kNN to need a relatively large k in this ~50-dimensional feature space, because the neighbourhoods are no longer truly "local" — all points are approximately equidistant in high dimensions.

```
In [94]: k_values = [3, 5, 7, 10, 15, 20, 30, 50, 75, 100, 150, 200]

knn_results = []
for k in k_values:
    pipe = Pipeline([
        ('pre', preprocessor),
        ('clf', KNeighborsClassifier(n_neighbors=k))
    ])
    scores = cross_val_score(pipe, x_train, y_train, cv=cv,
                              scoring='neg_log_loss', n_jobs=-1)

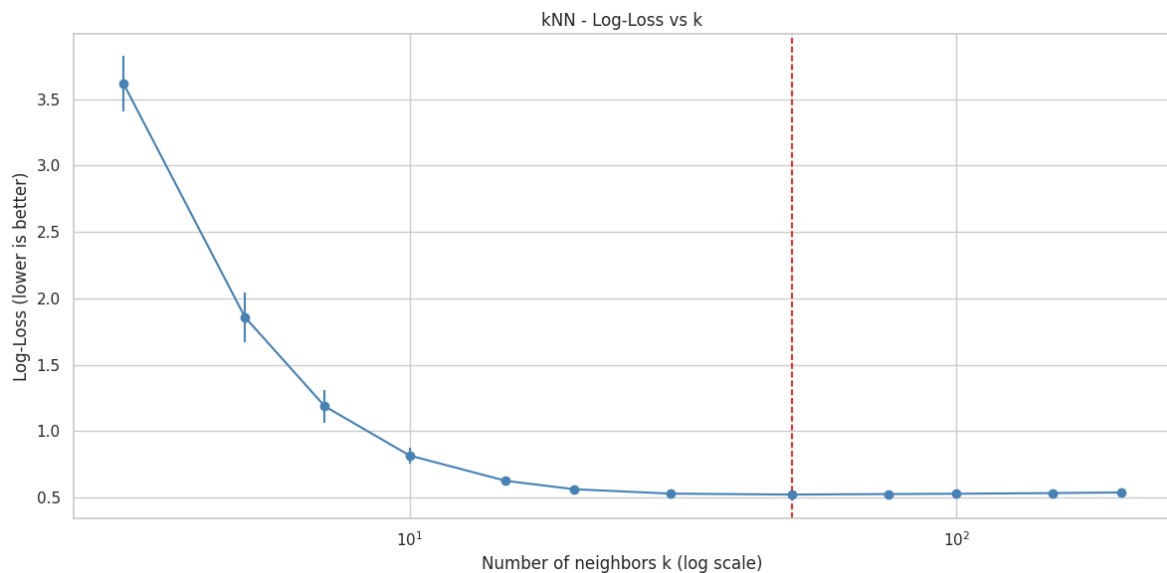
    knn_results.append({
        'k': k,
        'log_loss_mean': -scores.mean(),
        'log_loss_std': scores.std()
    })
    print(f"kNN (k={k}) - log-loss: {-scores.mean():.4f} ± {scores.std():.4f}")

knn_df = pd.DataFrame(knn_results)
best_k = knn_df.loc[knn_df['log_loss_mean'].idxmin(), 'k']
best_ll_knn = knn_df['log_loss_mean'].min()
print(f"\nBest kNN - k={best_k}, log-loss = {best_ll_knn:.4f}")

kNN (k=3) - log-loss: 3.6183 ± 0.2066
kNN (k=5) - log-loss: 1.8586 ± 0.1874
kNN (k=7) - log-loss: 1.1861 ± 0.1256
kNN (k=10) - log-loss: 0.8155 ± 0.0607
kNN (k=15) - log-loss: 0.6244 ± 0.0211
kNN (k=20) - log-loss: 0.5612 ± 0.0215
kNN (k=30) - log-loss: 0.5281 ± 0.0123
kNN (k=50) - log-loss: 0.5208 ± 0.0099
kNN (k=75) - log-loss: 0.5247 ± 0.0091
kNN (k=100) - log-loss: 0.5270 ± 0.0072
kNN (k=150) - log-loss: 0.5321 ± 0.0064
kNN (k=200) - log-loss: 0.5366 ± 0.0065
```

Best kNN - k=50, log-loss = 0.5208

```
In [100]: fig, ax = plt.subplots(figsize=(12, 6))
ax.errorbar(knn_df['k'], knn_df['log_loss_mean'], yerr=knn_df['log_loss_std'],
            ax.set_xscale('log')
ax.axvline(best_k, ls='--', color='red', lw=1.2, label=f'Best k = {best_k}')
ax.set_xlabel('Number of neighbors k (log scale)')
ax.set_ylabel('Log-Loss (lower is better)')
ax.set_title('kNN - Log-Loss vs k')
plt.tight_layout()
plt.show()
```



As anticipated, the optimal K is relatively large. Small k overfits, due to high variance from noisy estimates, while large k underfits. This shows us that the large dimensional space isn't truly local, and the model averages over many points to get stable estimates. This negates the flexibility that makes kNN attractive in lower dimensions.

LDA and QDA

```
In [101]: # LDA (no hyperparameters to tune) - just evaluate log-loss
lda_pipe = Pipeline([
    ('pre', preprocessor),
    ('clf', LinearDiscriminantAnalysis())
])
lda_scores = cross_val_score(lda_pipe, x_train, y_train, cv=cv, scoring='log_loss')
lda_log_loss_mean = -lda_scores.mean()
lda_log_loss_std = lda_scores.std()

print(f"LDA - log-loss: {lda_log_loss_mean:.4f} ± {lda_log_loss_std:.4f}")

# QDA (with reg_param tuning)
reg_params = [0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 1.0]
qda_results = []

for reg in reg_params:
    pipe = Pipeline([
        ('pre', preprocessor),
        ('clf', QuadraticDiscriminantAnalysis(reg_param=reg))
    ])
    scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_log_loss')
    qda_results.append({
        'reg_param': reg,
        'log_loss_mean': -scores.mean(),
        'log_loss_std': scores.std()
    })
    print(f"QDA (reg_param={reg}) - log-loss: {-scores.mean():.4f} ± {scores.std():.4f}")

qda_df = pd.DataFrame(qda_results)
best_reg = qda_df.loc[qda_df['log_loss_mean'].idxmin(), 'reg_param']
best_ll_qda = qda_df['log_loss_mean'].min()
print(f"\nBest QDA - reg_param={best_reg}, log-loss = {best_ll_qda:.4f}")
```

```
if -lda_log_loss_mean < best_ll_qda:  
    print("\nLDA performs better than QDA with any regularization.")  
else:  
    print("\nQDA with regularization performs better than LDA.")
```

LDA - log-loss: 0.5872 ± 0.0071

```

-----
ValueError                                Traceback (most recent call last)
Cell In[101], line 21
     16 for reg in reg_params:
     17     pipe = Pipeline([
     18         ('pre', preprocessor),
     19         ('clf', QuadraticDiscriminantAnalysis(reg_param=reg))
     20     ])
--> 21     scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_log_loss', n_jobs=-1)
     22     qda_results.append({
     23         'reg_param': reg,
     24         'log_loss_mean': -scores.mean(),
     25         'log_loss_std': scores.std()
     26     })
     27     print(f"QDA (reg_param={reg}) - log-loss: {-scores.mean():.4f} ± {scores.std():.4f}")

File ~/venvs/islp/lib/python3.12/site-packages/sklearn/utils/_param_validation.py:218, in validate_params.<locals>.decorator.<locals>.wrapper(*args, **kwargs)
    212 try:
    213     with config_context(
    214         skip_parameter_validation=(
    215             prefer_skip_nested_validation or global_skip_validation
n
    216         )
    217     ):
--> 218     return func(*args, **kwargs)
    219 except InvalidParameterError as e:
    220     # When the function is just a wrapper around an estimator, we
allow
    221     # the function to delegate validation to the estimator, but we
replace
    222     # the name of the estimator by the name of the function in the
error
    223     # message to avoid confusion.
    224     msg = re.sub(
    225         r"parameter of \w+ must be",
    226         f"parameter of {func.__qualname__} must be",
    227         str(e),
    228     )

File ~/venvs/islp/lib/python3.12/site-packages/sklearn/model_selection/_validation.py:651, in cross_val_score(estimator, X, y, groups, scoring, cv, n_jobs, verbose, params, pre_dispatch, error_score)
    648 # To ensure multimetric format is not supported
    649 scorer = check_scoring(estimator, scoring=scoring)
--> 651 cv_results = cross_validate(
    652     estimator=estimator,
    653     X=X,
    654     y=y,
    655     groups=groups,
    656     scoring={"score": scorer},
    657     cv=cv,
    658     n_jobs=n_jobs,
    659     verbose=verbose,
    660     params=params,

```

```

661     pre_dispatch=pre_dispatch,
662     error_score=error_score,
663 )
664 return cv_results["test_score"]

```

File ~/.venvs/islp/lib/python3.12/site-packages/sklearn/utils/_param_validation.py:218, in validate_params.<locals>.decorator.<locals>.wrapper(*args, **kwargs)

```

212 try:
213     with config_context(
214         skip_parameter_validation=(
215             prefer_skip_nested_validation or global_skip_validation
n
216         )
217     ):
--> 218         return func(*args, **kwargs)
219 except InvalidParameterError as e:
220     # When the function is just a wrapper around an estimator, we
allow
221     # the function to delegate validation to the estimator, but we
replace
222     # the name of the estimator by the name of the function in the
error
223     # message to avoid confusion.
224     msg = re.sub(
225         r"parameter of \w+ must be",
226         f"parameter of {func.__qualname__} must be",
227         str(e),
228     )

```

File ~/.venvs/islp/lib/python3.12/site-packages/sklearn/model_selection/_validation.py:393, in cross_validate(estimator, X, y, groups, scoring, cv, n_jobs, verbose, params, pre_dispatch, return_train_score, return_estimator, return_indices, error_score)

```

372 parallel = Parallel(n_jobs=n_jobs, verbose=verbose, pre_dispatch=p
re_dispatch)
373 results = parallel(
374     delayed(_fit_and_score)(
375         clone(estimator),
(...) 390     for train, test in indices
391 )
--> 393 _warn_or_raise_about_fit_failures(results, error_score)
395 # For callable scoring, the return type is only know after calling
. If the
396 # return type is a dictionary, the error scores can now be inserte
d with
397 # the correct key.
398 if callable(scoring):

```

File ~/.venvs/islp/lib/python3.12/site-packages/sklearn/model_selection/_validation.py:479, in _warn_or_raise_about_fit_failures(results, error_score)

```

472 if num_failed_fits == num_fits:
473     all_fits_failed_message = (
474         f"\nAll the {num_fits} fits failed.\n"
475         "It is very likely that your model is misconfigured.\n"
476         "You can try to debug the error by setting error_score='rai
ise'.\n\n"
477         f"Below are more details about the failures:\n{fit_errors_
summary}"

```

```

478     )
--> 479     raise ValueError(all_fits_failed_message)
481 else:
482     some_fits_failed_message = (
483         f"\n{num_failed_fits} fits failed out of a total of {num_f
its}.\n"
484         "The score on these train-test partitions for these parame
ters"
485         (...)
488         f"Below are more details about the failures:\n{fit
_errors_summary}"
489     )

```

ValueError:

All the 10 fits failed.

It is very likely that your model is misconfigured.

You can try to debug the error by setting `error_score='raise'`.

Below are more details about the failures:

```

-----
-----
10 fits failed with the following error:
Traceback (most recent call last):
  File "/home/grahamdib/.venvs/islp/lib/python3.12/site-packages/sklearn/m
odel_selection/_validation.py", line 833, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/home/grahamdib/.venvs/islp/lib/python3.12/site-packages/sklearn/b
ase.py", line 1336, in wrapper
    return fit_method(estimator, *args, **kwargs)
    ~~~~~^~~~~~

  File "/home/grahamdib/.venvs/islp/lib/python3.12/site-packages/sklearn/p
ipeline.py", line 621, in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/home/grahamdib/.venvs/islp/lib/python3.12/site-packages/sklearn/b
ase.py", line 1336, in wrapper
    return fit_method(estimator, *args, **kwargs)
    ~~~~~^~~~~~

  File "/home/grahamdib/.venvs/islp/lib/python3.12/site-packages/sklearn/d
iscriminant_analysis.py", line 1132, in fit
    raise linalg.LinAlgError(
numpy.linalg.LinAlgError: The covariance matrix of class 0 is not full ran
k. Increase the value of `reg_param` to reduce the collinearity.

```